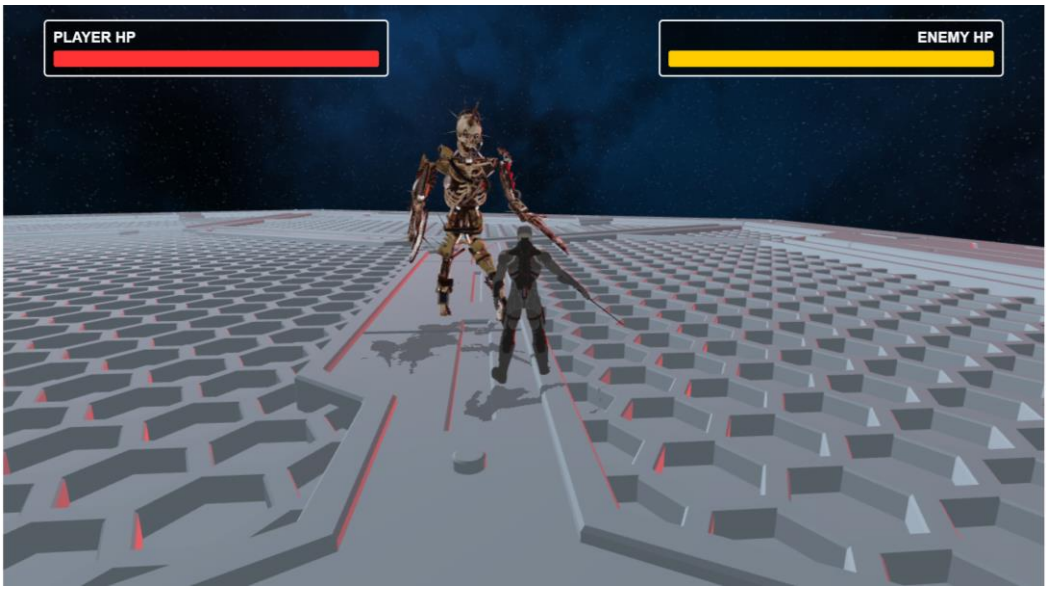
Computer Graphics Final Project Design Document

Basic Information

- **Game Title:** Virtual Border: The Last Compile
- **Personal Information:** 41347028S, Department of Computer Science and Information Engineering 117, Sophomore (Year 2), 吳舒萍



1. Game Concept

1.1 Game Type and Core Background

This game is a 3D Cyberpunk-style action game developed using pure native WebGL. The game takes place on a crashing virtual firewall border called "The Last Compile." The player acts as the last surviving defense program in the network. In an alien space, the player must fight a huge boss (Enemy) that is completely controlled by a bad virus.

1.2 Game Goal

The player's main goal is to defeat the virus monster using smart moving and good attacking rhythm. Before the player's own core breaks (PLAYER HP reaches 0), the player must swing the sword to empty the enemy's life bar (ENEMY HP). This allows the player to take back "The Root Key"—which saves human civilization—and successfully restart the system.

1.3 Intended Player Experience

We want to give players a fight experience that is smooth (no lag), deeply immersive, and has strong geometric feedback:

- **High Smoothness:** To make the movement fluid, the system uses texture caching and parallel loading. This stops the CPU from freezing and the network from lagging when loading complex multi-joint models. High-frequency WASD movement and zooming stay at a high frame rate, ensuring instant response.
- **Sci-Fi World Immersion:** We use different graphics techniques to build a deep cyber space. This includes a far-away digital matrix skybox, two-pass Shadow Mapping, and tangent-space Normal Mapping to show small bumps and details on the robot shell.
- **Data and Visual Hit Feedback:** I made a strict Euclidean distance collision check (distance < 0.7). When the player hits the enemy, the system reduces the enemy's HP bar and sends a signal to the Fragment Shader. This triggers a bright red neon flashing effect on the enemy's surface. This mix of geometry check, UI data, and shader color-changing gives a great sense of hitting.

2. Scene and Visual Design

2.1 Reasons for World Setting and Object Choices

- **Skybox:** The game is set in a virtual, alien world. I want to use a dark starry sky to make players feel like they are standing in a vast, empty space.
- **Platform:** To give characters a place to stand and to limit the battle area, I placed a platform. I chose a sci-fi style to match the sky background. At first, the scene looked too boring with only black, gray, and white. So, I changed the platform's color to gray-blue to add some color while keeping the metallic feel.
- **Reflective Crystal:** This object represents the data center of the system. I chose a crystal shape to make good use of Dynamic Cube

Map reflection. When the player and the big enemy fight near it, the moving reflections on the crystal show the battle between the two digital powers.

- **Player Character:** I chose a human-like model with a sci-fi style because it is easier for players to feel like they are the character. The weapon is a sword. A close-range weapon like a sword forces players to get close to the enemy, which creates risk, danger, and a more exciting experience. The 3D model is made of separate metal joint parts (body, upper arm, forearm, hand) to make the motion look more real.
- **Enemy Character:** I chose a monster model to create a clear good-vs-evil look against the player. Its 3D model is also made of separate metal joints (body, arms, legs, feet) to allow for lively movements.

2.2 Visual Style and Feelings

The visual style focuses on a strong contrast between the cold digital matrix and hot neon danger. I used a three-point lighting setup to guide the player's emotions:

- **Cold and Warm Contrast:** The background and dark sides use a dim blue Ambient Light to show that the cyber border is cold and lonely. A bright white Point Light shines from high above to create a bright center stage. This contrast guides the player’s eyes to the center of the battle.
- **Dynamic Colors for Danger:** A red Rim Light stays on the back of the characters, matching the neon red flash when a character gets hit. In psychology, red means system warning and overload. This constant red edge light gives players a feeling of cyber crisis.

3. Technical Choices and Parameters

3.1 Camera and Projection Matrix Settings

- **Camera Position and Control:** The camera starts at a set distance from the player (Orbit/Zoom camera). It works like modern 3D games: move the mouse to rotate the camera, and use the wheel to zoom. For

easy control, WASD movement is calculated dynamically based on the current horizontal direction of the camera.

- **FOV (Field of View) = 45 Degrees:** Compared to wide-angle lenses, 45 degrees keeps the models looking natural without bad distortions at the edge of the screen. This helps players judge the distance between themselves and the enemy.
- **Aspect Ratio = 16:9 (1280x720):** This is the gold standard for modern screens. It gives a wide view so players can see the enemy's wide attacks.
- **Near = 0.1, Far = 150.0:** Setting Near to 0.1 prevents screen clipping when the camera is very close to objects. Setting Far to 150.0 matches the skybox limit and keeps the Z-Buffer accurate, avoiding grid flickering (Z-Fighting).

3.2 Lighting Configuration

- **Light Numbers and Types:** The scene has one high-intensity Point Light and one weak Ambient Light.
- **Light Colors:** The ambient light is dim blue for a cold theme. The main light is bright white to keep the original texture colors true and create a spotlight effect. I also added a constant neon red directional light as a **Rim Light**. It outlines the edges of the models, helping players see the robot joints clearly against the dark background.
- **Light Position:** The point light is placed high up on one side. This asymmetric setting creates changing long shadows on the floor when the big enemy moves, adding a cool, high-pressure atmosphere.

3.3 Automated Material Recovery (MTL Parsing)

To avoid dull colors from hard-coded numbers, I wrote an automated MTL parsing function in loadModel.js. When the system loads an object, it reads the .mtl file, extracts the original light values (K_a, K_d, K_s), and sends them to the **Phong Shading** shader uniforms automatically.

3.4 Graphics Contribution of 3D Models and Textures

- **Low-Poly Models:** Low-poly models run very fast. By rotating their joint matrices, they create clean and sharp animations.
- **Tangent-Space Normal Mapping:** To add details to low-poly models without adding more faces, the system calculates the Tangent and Bitangent vectors at the vertices to build a TBN Matrix. The Fragment Shader reads the normal map to show fine scratches and screws through clever light and shadow calculations.
- **Two-Pass Shadow Mapping:**
    - *First Pass (Off-screen):* The camera moves to the light position to render a depth map (Shadow Map).
    - *Second Pass (On-screen):* The system compares the pixel position with the depth map to see if it is in shadow. This removes the "floating" look and makes characters feel grounded.
- **Dynamic Environment Reflection & Refraction:** The crystal in the center uses the view vector and surface normal to calculate reflection and refraction. The refraction ratio is set to 1.00/1.15 (air to glass). It mixes them using mix(colorReflect, colorRefract, 0.5) to simulate real glass optics.

4. Challenges and Optimizations

4.1 Pre-loading Optimization for Multi-Joint Models

- **The Problem:** The player and enemy are made of many separate joint meshes. At first, each joint part fetched the same character.mtl file over the network. This caused dozens of duplicate requests and heavy CPU work, making the game freeze during loading.
- **The Solution:** I updated the loading pipeline in player.js:
    1. *Material Caching:* The top-level script fetches and reads character.mtl only once and saves it as a shared Mtl Package.

2. *Decoupled Mesh Fetching:* This package is passed into each joint's loadModel function. The system skips duplicate downloads and only loads the .obj shape data.
3. *Promise Pipelining:* I used JavaScript Promise.all() to load all parts at the same time in parallel.

4.2 Mesh Separation and the Flight Choice

- **The Challenge:** Most free .obj assets combine all body parts into one file. I used Blender to cut the arms, body, and legs into separate files so they can rotate using matrices.
- **Design Compromise (Floating Setting):** The player's legs and lower body were fused together in the original mesh. Cutting them apart caused bad holes in the model. Because time was limited, I made a creative choice: I removed the leg walking animation and changed the player's movement to "hovering/flying." This fits the cyber theme perfectly and avoids stiff leg looks.
- **Pivot Point Reflection:** Moving the pivot points (origin) of joints in Blender was tricky. The enemy's pivot points are highly accurate, so its attacks look natural. The player's pivot points have small errors, making the sword swinging look a bit less smooth. This was a great learning experience.

4.3 Keyframe Interpolation System

- **The Challenge:** Writing smooth animations by hand in WebGL is hard. I did not use simple angle addition. Instead, I built an interpolation engine.
- **How It Works:** For actions like WALK or ATTACK, we set a "Start Pose matrix" and an "End Pose matrix." In the main render loop, a function calculates the smooth transition step for each frame. This makes the character change poses smoothly without sudden jumps.

5. Audio Design

5.1 Music Selection and Theme Fitting

The music selection matches the cyberpunk crisis and digital battle theme perfectly:

- **Main Menu BGM: "aLIEz"**
    - *Reason:* This famous song by Hiroyuki Sawano starts with low electronic tones and explodes into heavy rhythms. It fits the feeling of a collapsing system and a final world border.
- **Battle BGM: "Radiance Across Sea and Sky" (Wuthering Waves)**
    - *Reason:* This track mixes electronic beats with a grand orchestra. It is fast and full of energy, helping players focus during fast movements, dodges, and sword fights.

5.2 Audio State Machine Transitions

The audio controller is connected to the game's logic states:

- *Main Menu:* Plays **"**aLIEz**"** on a loop to set a tense theme.
- *Intro Panel:* **"**aLIEz**"** continues playing during the text animation.
- *Real Game:* When enterRealGame() is called, the system stops the menu music instantly and starts the high-BPM battle track to boost the player's adrenaline.
- *Game Over:* If playerHp === 0, stopallBGM() is called immediately to turn off the music, creating a sudden silence that delivers a strong impact of defeat.

5.3 Copyright and Disclaimer

Because of copyright laws, this project **cannot be used for public commercial purposes or public distribution**. If this project is uploaded online for portfolio display, the background music will be muted or replaced with copyright-free audio to respect intellectual property rights.

6. UI and Control Flow

6.1 HTML/CSS and WebGL Canvas Layers

The project overlays efficient HTML5/CSS3 2D elements on top of the 3D WebGL Canvas:

- **Dual HUD HP Bars:** The PLAYER HP and ENEMY HP bars are placed at the top corners. They use a semi-transparent black background with a neon red fill. The CSS uses transition: width 0.1s ease-out; so that the health bars shrink smoothly when a character gets hurt.

6.2 Story Intro and Menu State Machine

To tell the story background, I built a smooth game menu state machine using native web tools:

1. **Main Menu State:** The game starts with a static 3D scene in the background, covered by a neon title and control buttons.
2. **Intro Panel State:** When the player clicks start, a 35-second Star Wars-style scrolling text animation shows up. We use CSS3 @keyframes to make the text tilt and move away in 3D space. The system listens for the animationend event, or lets the player click to skip, and smoothly calls enterRealGame().
3. **Battle and Result State:** Once in the game, all menu panels are hidden, showing the health bars and unlocking full mouse camera controls. When the battle ends, the system displays #win-screen or #lose-screen with pulsing neon text animations ("YOU WIN" in cyan or "YOU LOSE" in red) to finish the game loop.